Payment.java

Line-by-Line Code Explanation

```
public abstract class Payment {
```
''' Defines the base template for all payments. "abstract" means you cannot create a generic Payment object directly; you must create a specific child class.

```
private static int ticketCounter = 1000;
```
''' Creates a global counter shared across all payments. "static" means it belongs to the class itself, not individual objects. We use this to generate unique, incrementing IDs.

```
protected String paymentId;
protected double amount;
protected String status;
```
''' Declares variables to hold the transaction data. "protected" means they are hidden from outside classes (Encapsulation), but accessible to child classes (like PayPal or CreditCard).

```
public Payment(double amount, String currency) {
```
''' The Constructor method. This runs automatically the moment any child payment object is created using the "new" keyword.

```
if (amount <= 0) { throw new IllegalArgumentException(...); }
```
''' A validation check to ensure the system rejects negative or zero dollar transactions immediately.

```
this.paymentId = "TXN" + (ticketCounter++);
```
''' Generates the unique ID (e.g., "TXN1000") and then immediately increases the counter (++) so the next payment gets "TXN1001".

```
this.amount = amount;
this.currency = currency;
this.status = "PENDING";
```
''' Saves the provided data into the object's memory and sets the initial state of the payment to PENDING before it is processed.

```
this.timestamp = LocalDateTime.now();
```
''' Captures the exact current date and time from the system clock.

```
public abstract boolean process(BiConsumer<String, String> log);
```
''' The most critical line. "abstract" means there is no code here. It forces every child class to write their own custom "process()" method. The BiConsumer parameter allows them to send live log messages back to the UI.

```
public String getPaymentId() { return paymentId; }
```
''' Getter methods safely allow outside classes (like the PaymentProcessor) to read the protected variables without being able to change them.